Assert.js v1.2.0 cheatsheet - https://github.com/Serrin/assert.js

Category	Details	
Import assert	`import assert from "./assert.js";` `globalThis.assert = assert;`	
Import assert as default	`import { default as assert } from "./assert.js"; globalThis.assert = assert;`	
Import assert as defaultExport	`import defaultExport from "./assert.js";` `globalThis.assert = defaultExport;`	
Dynamic import	`const assert = await import("./assert.js"); globalThis.assert = defaultExport;`	
Config	`assert.config.alwaysStrict;` Default value is false. If value is true, then `assert.equal();` will be replaced with `assert.strictEqual();` and `assert.notEqual();` will be replaced with the `assert.notStrictEqual();`.	

Testrunner functions	`type TestResult<T> =`
	`  | {ok: true, value: T, block: Function, name: string}`
	`  | {ok: false, error: Error, block: Function, name: string};`	

	`assert.testSync(name = "assert.testSync", block): TestResult`
	`assert.test(name = "assert.testSync", block): TestResult`
	`assert.it(name = "assert.testSync", block): TestResult`	

	`await assert.testAsync(name = "assert.testAsync", block): TestResult`	
	`assert.testCheck(result: TestResult): result.ok is true`	
class assert.TestSuite	`Add(...args: Array<TestResult>): this;`	`success(): IterableIterator<TestResult>;`
	`clear(): this;`	`failed(): IterableIterator<TestResult>;`
	`get size(): number;`	`values(): IterableIterator<TestResult>;`
	`toArray(): Array<TestResult>`	`[Symbol.iterator]()`
Argument types	`type message = string | Error;`	
	`type ExpectedType = string | function | Array<string | function>;`	
Constants	`assert.VERSION;`	
Errors (class)	`assert.AssertionError([options: {[message], [cause], [actual], [expected], [operator] }]);`	
Exceptions	`assert.throws(fn,[ErrorType | string | RegExp][,message]): Promise;`	
	`await assert.rejects(asyncFnOrPromise,[ErrorType | string | RegExp][,message]);`	
	`await assert.doesNotReject(asyncFnOrPromise,[ErrorType | string | RegExp][,message]);`	
Basic	`assert(condition[,message]);`	`assert.fail([message]);` `assert.fail([actual,expected,message:string,operator]);`
	`assert.ok(condition[,message]);`	`assert.notOk(condition[,message]);`
Equality	`assert.equal(actual,expected[,message]);`	`assert.notEqual(actual,expected[,message]);`
	`assert.strictEqual(actual,expected[,message]);`	`assert.notStrictEqual(actual,expected[,message]);`
	`assert.deepEqual(actual,expected[,message]);` `assert.deepStrictEqual(actual,expected[,message]);`	`assert.notDeepEqual(actual,expected[,message]);` `assert.notDeepStrictEqual(actual,expected[,message]);`
	`assert.oneOf(value,collection[,message]);`	`assert.notOneOf(value,collection[,message]);`

Comparasion	assert.lt(value1, value2[,message]);	assert.lte(value1,value2[,message]);
	assert.gt(value1, value2[,message]);	assert.gte(value1,value2[,message]);
	assert.inRange(value,min,max[,message]);	assert.notInRange(value,min,max[,message]);
String	assert.match(string,regexp[,message]);	assert.doesNotMatch(string,regexp[,message]);
	assert.stringContains(actual,substring[,message]);	assert.stringNotContains(actual,substring[,message]);
	assert.stringStartsWith(actual,substring[,message]);	assert.stringNotStartsWith(actual,substring[,message]);
	assert.stringEndsWith(actual,substring[,message]);	assert.stringNotEndsWith(actual,substring[,message]);
Object	assert.includes(container,{keyOrValue,[value]} [,message]);	assert.doesNotInclude(container,{keyOrValue,[value]} [,message]);
	assert.isEmpty(value[,message]);	assert.isNotEmpty(value[,message]);
Type	assert.is(value, expectedType: ExpectedType [,message]);	assert.isNot(value, expectedType: ExpectedType [,message]);
	assert.typeOf(value, expectedType: string [,message]);	assert.notTypeOf(value, expectedType: string [,message]);
	assert.instanceOf(value, expectedType: function [,message]);	assert.notInstanceOf(value, expectedType: function [,message]);
	assert.isPrimitive(value[,message]);	assert.isNotPrimitive(value[,message]);
	assert.isNullish(value[,message]); assert.ifError(value[,message]);	assert.isNonNullable(value[,message]);
	assert.isNull(value[,message]);	assert.isNotNull(value[,message]);
	assert.isUndefined(value[,message]);	assert.isDefined(value[,message]);
	assert.isNumber(value[,message]);	assert.isNotNumber(value[,message]);
	assert.isNaN(value[,message]);	assert.isNotNaN(value[,message]);
	assert.isInt(value[,message]);	assert.isNotInt(value[,message]);
	assert.isFloat(value[,message]);	assert.isNotFloat(value[,message]);
	assert.isString(value[,message]);	assert.isNotString(value[,message]);
	assert.isBigInt(value[,message]);	assert.isNotBigInt(value[,message]);
	assert.isBoolean(value[,message]);	assert.isNotBoolean(value[,message]);
	assert.isTrue(value[,message]);	assert.isNotTrue(value[,message]);
	assert.isFalse(value[,message]);	assert.isNotFalse(value[,message]);
	assert.isSymbol(value[,message]);	assert.isNotSymbol(value[,message]);
	assert.isFunction(value[,message]);	assert.isNotFunction(value[,message]);
	assert.isObject(value[,message]);	assert.isNotObject(value[,message]);